

Лабораторна робота №2. Використання СУБД PostgreSQL при створенні вебзастосунку мовою Python

Завдання. Для проєкта з попередньої лабораторної роботи провести міграцію даних на СУБД PostgreSQL зі збереженням базового функціоналу вебзастосунка та додаванням нового за рахунок застосування PostgreSQL в цілому, та бібліотеки Psycopg зокрема.

ЗМІСТ

Робота з базою даних PostgreSQL.....	3
1 Модуль psycopg. Підключення до сервера PostgreSQL	3
1.1 Встановлення psycopg	3
1.2 Підключення до сервера PostgreSQL.....	4
1.3 Курсор і операції з даними	5
1.4 Закриття підключення та курсору.....	6
1.5 Модель виконання запитів.....	7
2 Створення бази даних і таблиці в PostgreSQL	8
2.1 Створення бази даних	8
2.2 Створення таблиці	10
3 Додавання даних у PostgreSQL.....	12
3.1 Додавання даних	12
3.2 Встановлення параметрів.....	12
3.3 Множинна вставка.....	14
4 Отримання даних із БД PostgreSQL	15
4.1 Отримання всіх рядків.....	16
4.2 Отримання частини рядків.....	17
4.3 Отримання одного рядка.....	19
5 Оновлення та видалення даних у PostgreSQL	20
5.1 Оновлення.....	20
5.2 Видалення даних.....	23

Робота з базою даних PostgreSQL

1 Модуль psycopg. Підключення до сервера PostgreSQL

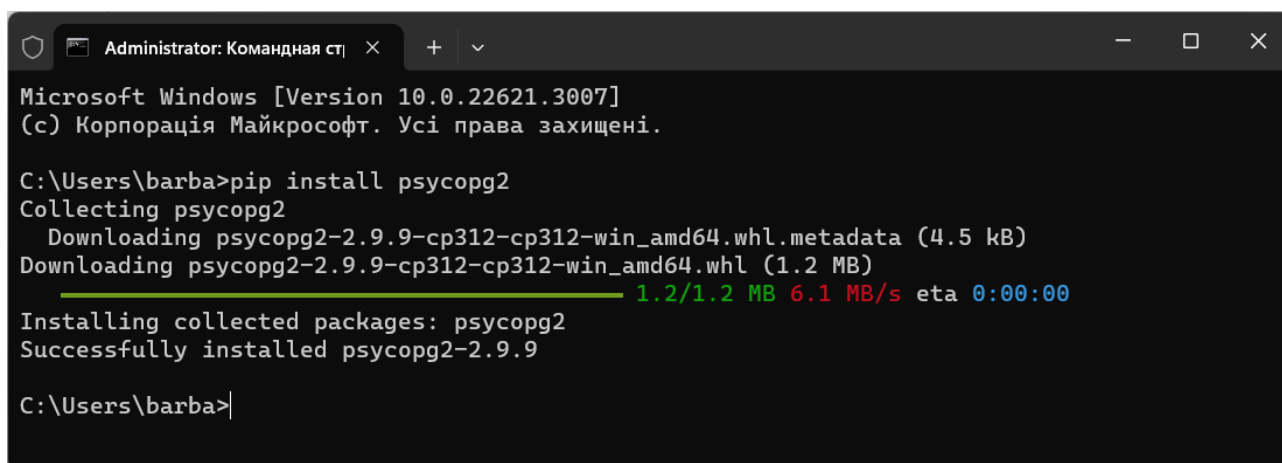
Однією з найбільш популярних реляційних систем баз даних є *PostgreSQL*. Розглянемо, як працювати з базами даних *PostgreSQL* у додатку мовою *Python*.

Перед початком роботи, природно, має бути встановлена сама *PostgreSQL*. Про встановлення *PostgreSQL* можна прочитати у відповідній статті Встановлення сервера *PostgreSQL*. Стандартні бібліотеки *Python* не надають вбудованого функціоналу для роботи з *PostgreSQL*, проте є велика кількість сторонніх бібліотек. Найпопулярнішою з них є *Psycopg 2* ([офіційний сайт Psycopg](#)). Ця бібліотека реалізована мовою C, завдяки чому має порівняно велику продуктивність.

1.1 Встановлення psycopg

Для встановлення виконаємо в терміналі таку команду:

```
pip install psycopg2
```



```
Administrator: Командная строка
Microsoft Windows [Version 10.0.22621.3007]
(c) Корпорация Майкрософт. Все права защищены.

C:\Users\barba>pip install psycopg2
Collecting psycopg2
  Downloading psycopg2-2.9.9-cp312-cp312-win_amd64.whl.metadata (4.5 kB)
  Downloading psycopg2-2.9.9-cp312-cp312-win_amd64.whl (1.2 MB)
    ━━━━━━━━━━━━━━━━━━━━━━━━━━━━━━━━━ 1.2/1.2 MB 6.1 MB/s eta 0:00:00
Installing collected packages: psycopg2
Successfully installed psycopg2-2.9.9

C:\Users\barba>
```

Рисунок 1 – Встановлення пакету psycopg2

Після цього ми можемо імпортувати бібліотеку в програми на *Python*:

```
import psycopg2
```

1.2 Підключення до сервера PostgreSQL

Для підключення до сервера *PostgreSQL* застосовується функція *connect()*. Вона приймає налаштування підключення:

```
psycopg2.connect(dbname="db_name", host="db_host", user="db_user",  
password="db_pass", port="db_port")
```

Функція приймає такі параметри:

- *dbname*: ім'я бази даних
- *user*: ім'я користувача
- *password*: пароль користувача
- *host*: хост/адреса сервера
- *port*: порт (якщо не вказано, то використовується порт за замовчуванням - 5432)

У разі вдалого підключення функція *connect* створює нову сесію бази даних і повертає об'єкт *connection*. Клас *connection* надає низку методів для роботи з підключенням до БД:

- *close()*: закриває підключення
- *cursor()*: повертає об'єкт *cursor* для здійснення запитів до БД
- *commit()*: підтверджує транзакцію
- *rollback()*: відкочує транзакцію

Наприклад, підключимося до стандартної бази даних "*postgres*" на локальному сервері *PostgreSQL*:

```
import psycopg2

conn = psycopg2.connect(dbname="postgres", user="postgres", password="masterkey",
host="127.0.0.1", port="5432")
print("Підключення встановлено")
conn.close()
```

У цьому випадку підключення йде для вбудованого користувача за замовчуванням "postgres".

1.3 Курсор і операції з даними

Метод *cursor()* об'єкта *connection* повертає курсор - об'єкт *cursor*, через який можна надсилати запити до бази даних. Для цього клас *cursor* надає низку методів:

- *execute(query, vars=None)*: виконує одну SQL-інструкцію. Через другий параметр у код SQL можна передати набір параметрів у вигляді списку або словника
- *execemany(query, vars_list)*: виконує параметризовану SQL-інструкцію. Через другий параметр приймає набори значень, які передаються у виконуваний код SQL.
- *callproc(procname[, parameters])*: виконує збережену функцію. Через другий параметр можна передати набір параметрів у вигляді списку або словника
- *mogrify(operation[, parameters])*: повертає код запиту SQL після прив'язки параметрів
- *fetchone()*: повертає наступний рядок з отриманого з БД набору рядків у вигляді кортежу. Якщо рядків у наборі немає, то повертає None
- *fetchmany([size=cursor.arraysize])*: повертає набір рядків у вигляді списку. кількість рядків, що повертаються, передається через параметр. Якщо більше рядків немає в наборі, то повертається порожній список.
- *fetchall()*: повертає всі (що залишилися) рядки у вигляді списку. За відсутності рядків повертається порожній список.
- *scroll(value[, mode='relative'])*: переміщує курсор у наборі на позицію *value* відповідно до режиму *mode*.

Визначення курсору:

```
import psycopg2

conn = psycopg2.connect(dbname="postgres", user="postgres", password="masterkey",
host="127.0.0.1")
cursor = conn.cursor()

cursor.close() # закриваємо курсор
conn.close() # закриваємо підключення
```

1.4 Закриття підключення та курсору

Варто зазначити, що обидва об'єкти - *connection* і *cursor* можуть використовуватися як менеджери контексту. Тобто за допомогою виразу *with* визначити контекст. Однак якщо об'єкт *cursor* по завершенню закривається, то об'єкт *connection* НЕ закривається:

```
import psycopg2

conn = psycopg2.connect(dbname="postgres", user="postgres", password="masterkey",
host="127.0.0.1", port="5432")
with conn:
    with conn.cursor() as cursor:
        print("Підключення встановлено")

print(cursor.closed) # True - курсор закритий
# cursor.close() # немає сенсу - об'єкт cursor уже закритий
conn.close() # об'єкт conn не закритий, треба закривати
```

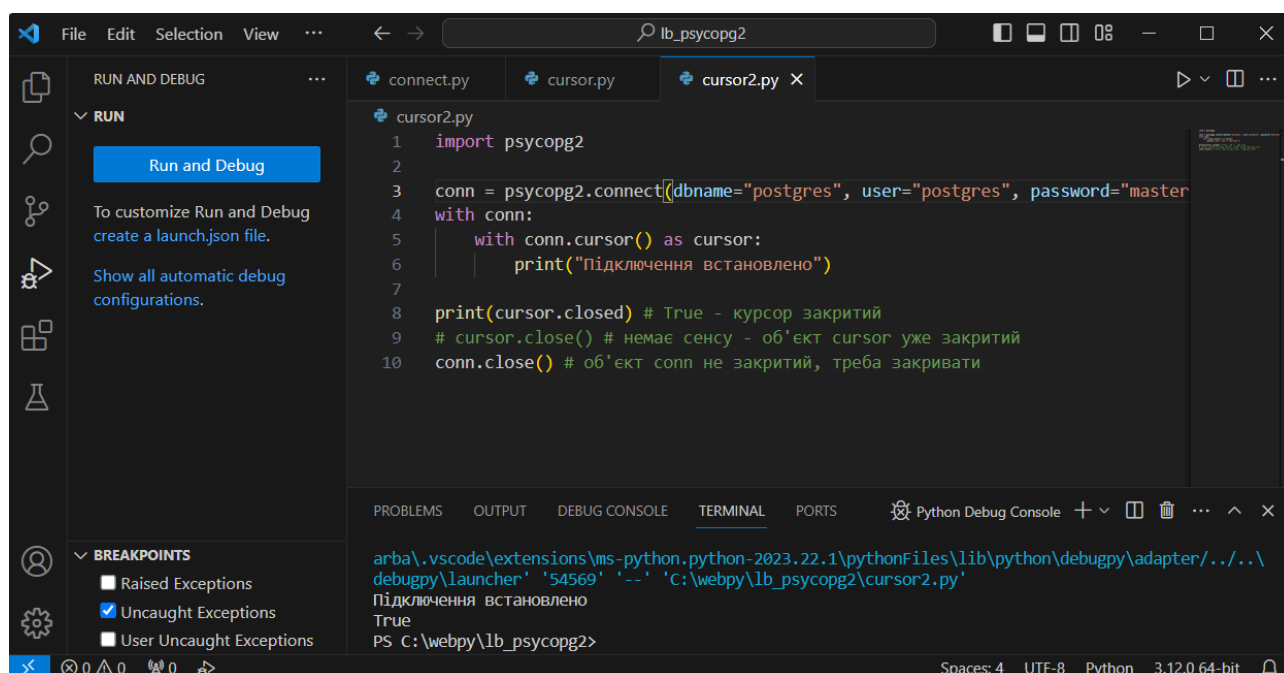


Рисунок 2 – Результат закриття підключення та курсору

1.5 Модель виконання запитів

Перед виконанням першої команди *SQL* автоматично створюється транзакція, у процесі якої можна виконувати різні вирази *SQL* за допомогою методів *execute/executemany* курсора, але для підтвердження їхнього виконання необхідно викликати метод *commit()* об'єкта *connection*. Умовно це може мати такий вигляд:

```
import psycopg2

conn = psycopg2.connect(dbname="postgres", user="postgres", password="masterkey",
                        host="127.0.0.1")
cursor = conn.cursor()

cursor.execute(sql1)

conn.commit() # реальне виконання команд sql1

cursor.close()
conn.close()
```

Тут реальне виконання умовної команди *sql1* відбувається тільки під час виконання методу *conn.commit()*. Якщо ж треба, щоб вирази *sql* автоматично виконувалися під час кожного виклику методу *cursor.execute()*, то можна встановити автокомміт за допомогою властивості *connection.autocommit*:

```
import psycopg2

conn = psycopg2.connect(dbname="postgres", user="postgres", password="masterkey",
                        host="127.0.0.1")

conn.autocommit = True # встановлюємо автокомміт

cursor = conn.cursor()
cursor.execute(sql1) # безпосереднє виконання команди sql1

cursor.close()
conn.close()
```

2 Створення бази даних і таблиці в PostgreSQL

2.1 Створення бази даних

Для створення бази даних застосовується *SQL-команда CREATE DATABASE*, якій передається ім'я бази даних. Наприклад, створимо базу даних з іменем "example":

```
import psycopg2

conn = psycopg2.connect(dbname="postgres", user="postgres", password="masterkey",
                        host="127.0.0.1")
cursor = conn.cursor()

conn.autocommit = True
# команда для створення бази даних metanit
sql = "CREATE DATABASE example"

# виконуємо код sql
cursor.execute(sql)
print("База даних успішно створена")

cursor.close()
conn.close()
```

Зверніть увагу, що для виразу "CREATE DATABASE" необхідно встановити автокомміт. Завдяки цьому команда *SQL*, по-перше, виконується негайно. А по-друге, виконується поза транзакцією (вираз "CREATE DATABASE" має виконуватися саме поза транзакцією).

```
conn.autocommit = True
```

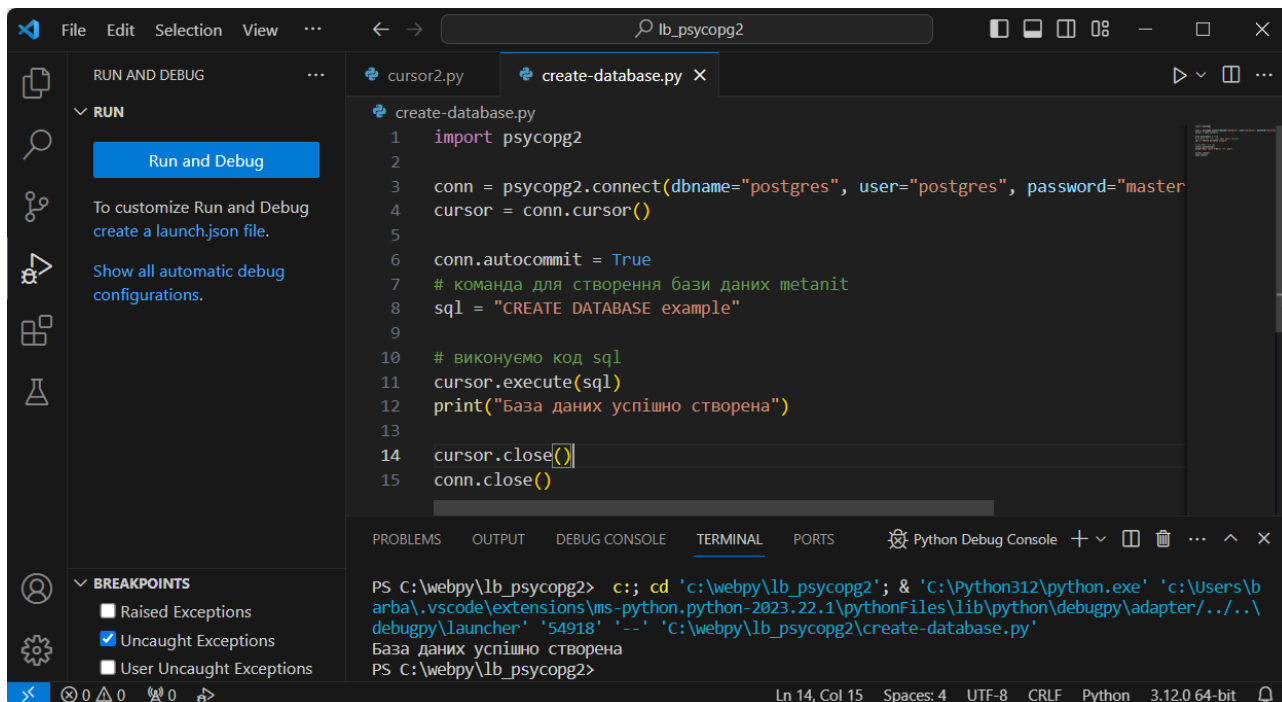



Рисунок 3 – Результат виконання скрипта

Після цього на сервері ми зможемо знайти базу даних "example".

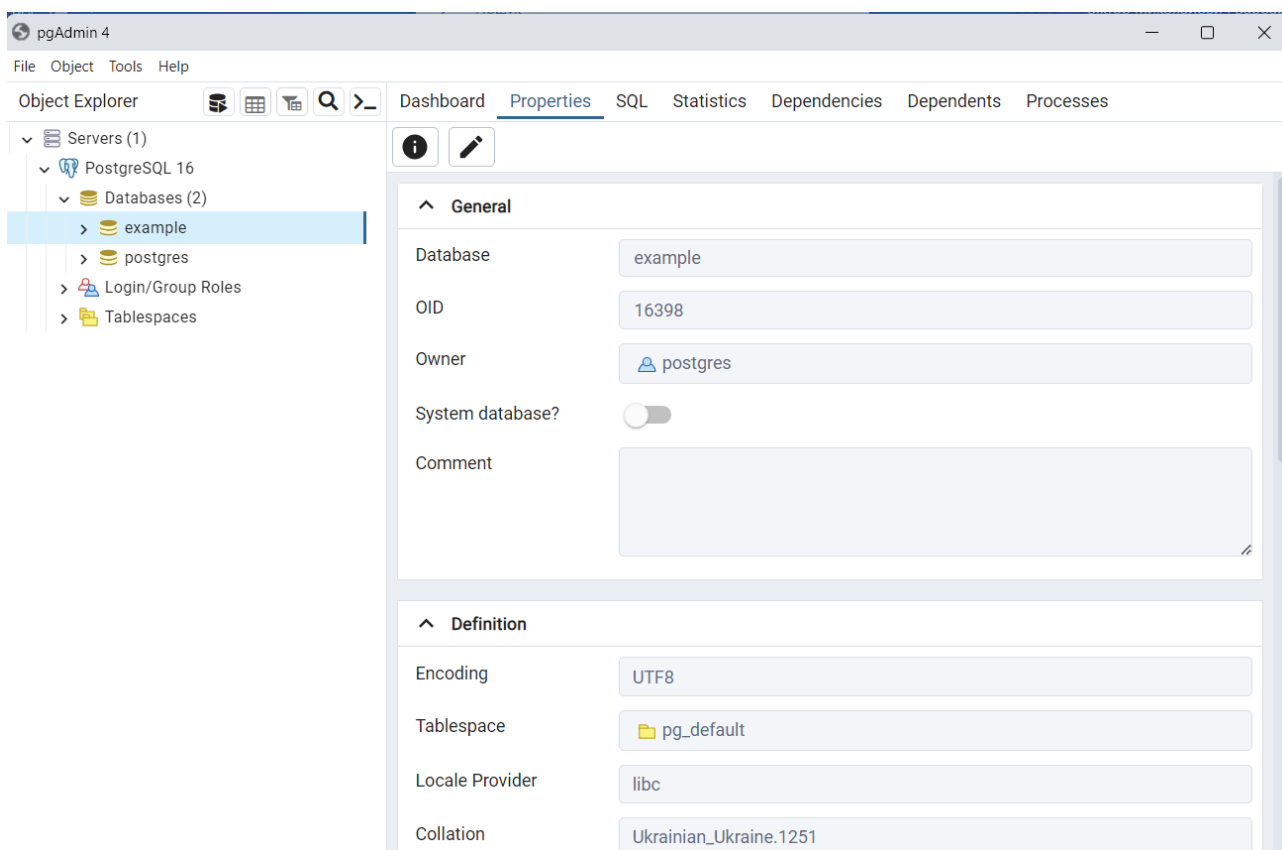


Рисунок 4 – Створення бази даних PostgreSQL в Python

2.2 Створення таблиці

Для створення таблиці в PostgreSQL застосовується інструкція `CREATE TABLE`. Наприклад, у виществореній базі даних "metanit" створимо таблицю `people`:

```
import psycopg2

conn = psycopg2.connect(dbname="example", user="postgres", password="masterkey",
                        host="127.0.0.1")
cursor = conn.cursor()

# створюємо таблицю people
cursor.execute("CREATE TABLE people (id SERIAL PRIMARY KEY, name VARCHAR(50), age
INTEGER)")
# підтверджуємо транзакцію
conn.commit()
print("Таблиця people успішно створена")

cursor.close()
conn.close()
```

У метод `cursor.execute()` передається інструкція `CREATE TABLE`, яка створює таблицю `people` з трьома стовпцями. Стовпець `id` представляє ідентифікатор користувача, зберігає дані типу *Serial*, тобто число, яке буде автоматично генеруватися та інкрементуватися з кожним новим рядком, і яке представляє первинний ключ. Другий стовпець - `name` представляє рядок - ім'я користувача. І третій стовпець - `age` представляє вік користувача.

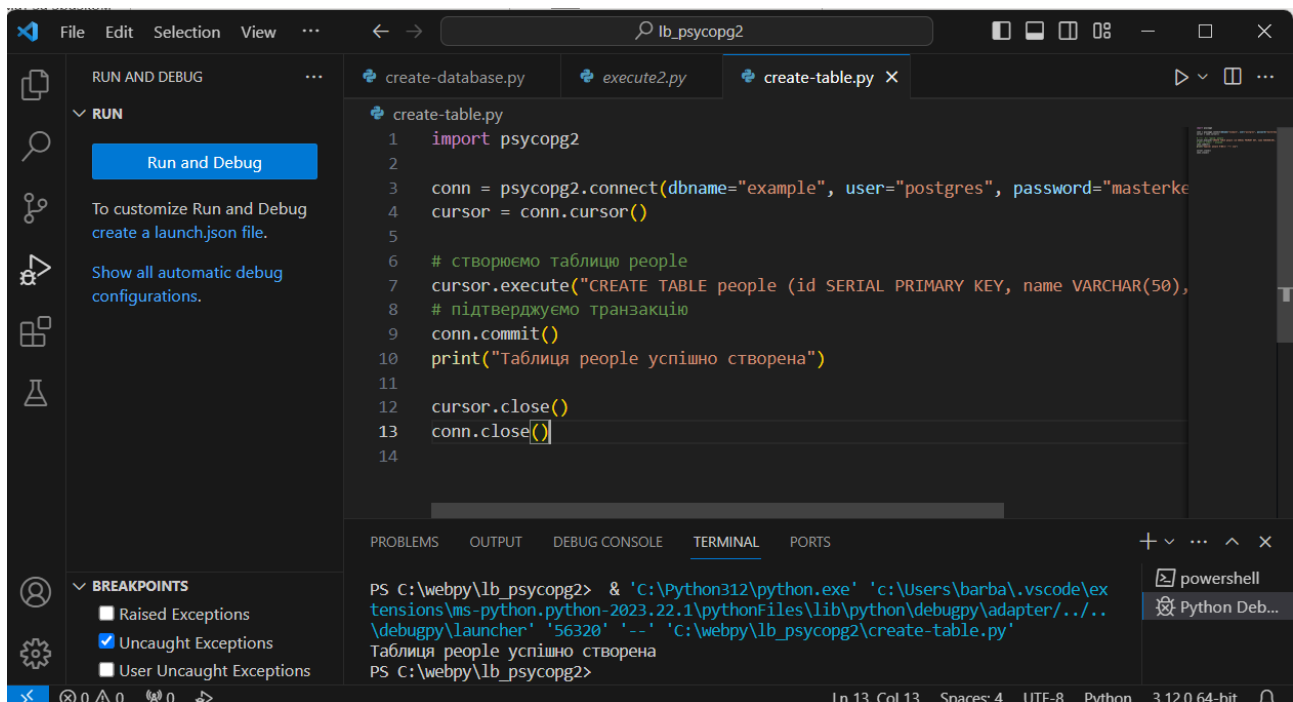


Рисунок 5 – Результат виконання скрипта

Після виконання скрипта ми можемо відкрити базу даних і побачити створену таблицю

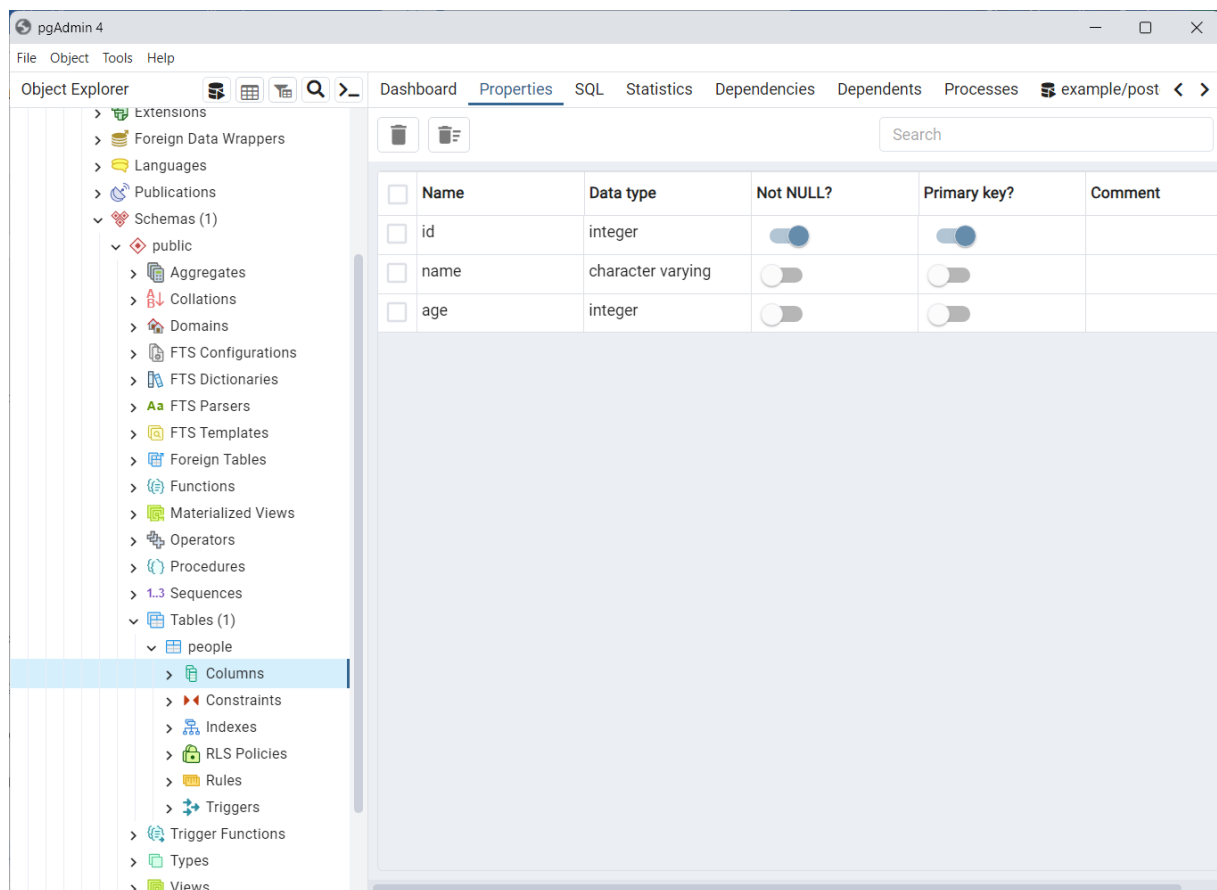


Рисунок 6 – Створення таблиці в БД PostgreSQL

3 Додавання даних у PostgreSQL

Розглянемо додавання в базу даних *PostgreSQL* на прикладі таблиці, створеної на минулому етапі (рис.6).

3.1 Додавання даних

Для додавання даних застосовується *SQL*-інструкція *INSERT*. Для додавання одного рядка використовуємо метод *execute()*:

```
import psycopg2

conn = psycopg2.connect(dbname="example", user="postgres", password="masterkey",
                        host="127.0.0.1")
cursor = conn.cursor()

# додаємо рядок в таблицю people
cursor.execute("INSERT INTO people (name, age) VALUES ('Quentin', 60)")
# виконуємо транзакцію
conn.commit()
print("Дані додано")

cursor.close()
conn.close()
```

Тут додається один рядок, де *name* = "Quentin", а *age* = 60. Перед виконанням команди *INSERT* відкривається транзакція, для завершення якої необхідно викликати метод *commit()* поточного об'єкта *Connection*.

3.2 Встановлення параметрів

За допомогою другого параметра в метод *execute()* можна передати значення для параметрів *SQL*-запиту:

```
import psycopg2

conn = psycopg2.connect(dbname="example", user="postgres", password="masterkey",
host="127.0.0.1")
cursor = conn.cursor()

# дані для додавання
jerome = ("Jerome", 45)
cursor.execute("INSERT INTO people (name, age) VALUES (%s, %s)", jerome)
conn.commit()

print("Дані додано")

cursor.close()
conn.close()
```

У цьому випадку значення, що додаються в БД, представляють кортеж `jerome`. У SQL-запиті замість конкретних значень використовуються знаки підстановки `%s`. Замість цих символів під час виконання запиту будуть вставлятися дані з кортежу `data`. Так, перший елемент кортежа - рядок `"Jerome"` передається на місце першого плейсхолдера `%s`, другий елемент - число `45` передається на місце другого плейсхолдера `%s`. Тобто в підсумку команда SQL матиме такий вигляд:

```
INSERT INTO people (name, age) VALUES (Jerome, 45)
```

Також зверніть увагу, що НЕ треба поміщати плейсхолдер `%s` у лапки - `psycopg2` робить це автоматично.

І якщо ми подивимося на вміст бази даних, то знайдемо там усі додані об'єкти:

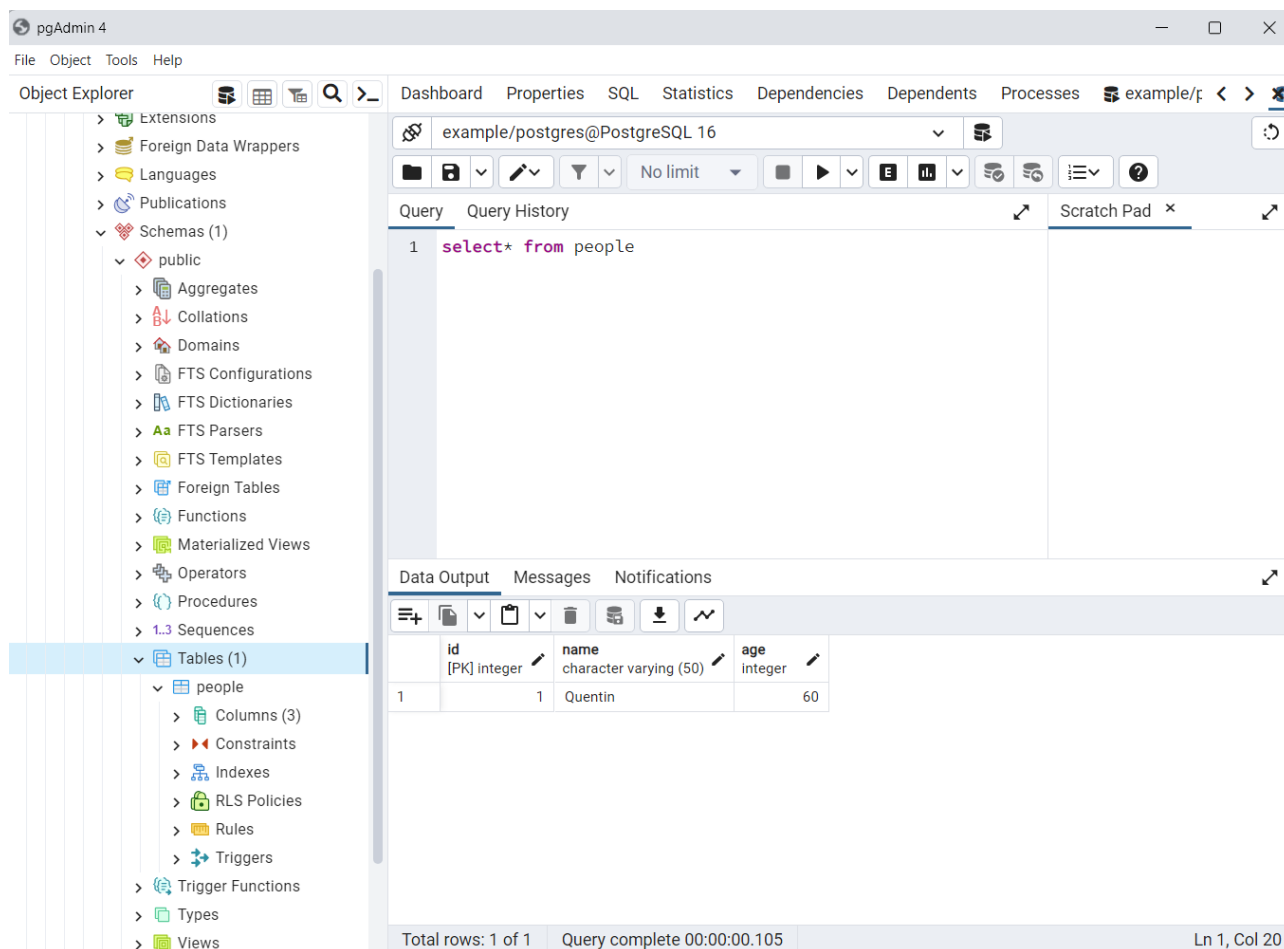


Рисунок 7 –Додавання даних до таблиці БД PostgreSQL

3.3 Множинна вставка

Метод `executemany()` дозволяє вставити набір рядків:

```
import psycopg2

conn = psycopg2.connect(dbname="example", user="postgres", password="masterkey",
                        host="127.0.0.1")
cursor = conn.cursor()

# дані для додавання
people = [("Samuel", 30), ("Leroy", 40), ("Jackson", 50)]
cursor.executemany("INSERT INTO people (name, age) VALUES (%s, %s)", people)

conn.commit()
print("Дані додано")

cursor.close()
conn.close()
```

У метод `cursor.executemany()`, по суті, передається той самий вираз *SQL*, тільки тепер дані визначені у вигляді списку кортежів *people*. Фактично кожен кортеж у цьому списку представляє окремий рядок - дані окремого користувача, і під час виконання методу для кожного кортежу створюватиметься свій вираз *INSERT INTO*

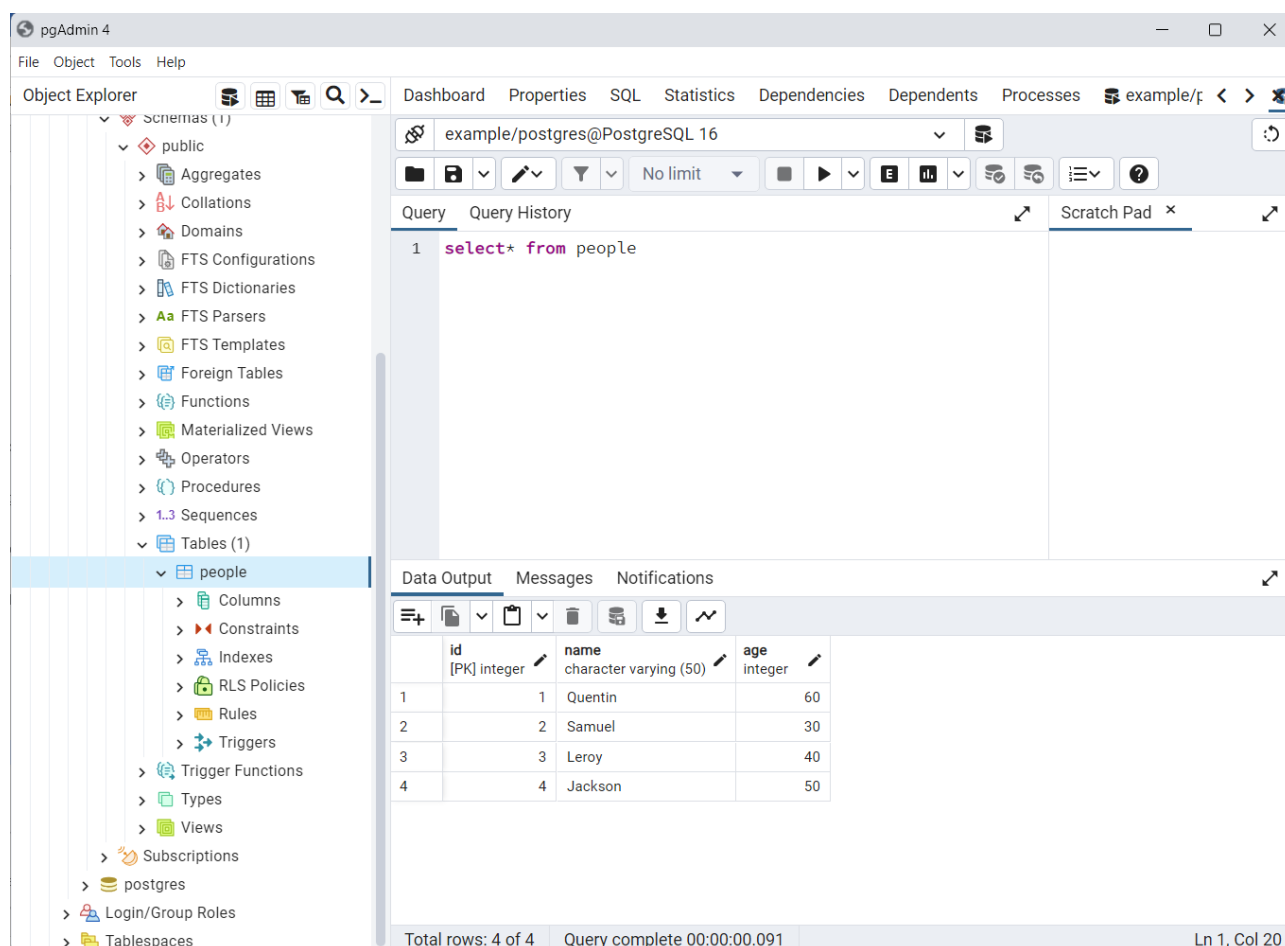


Рисунок 8 - Додавання безлічі рядків до таблиці БД PostgreSQL

4 Отримання даних із БД PostgreSQL

Для отримання даних застосовується *SQL*-команда *SELECT*. Після виконання цієї команди курсор отримує дані, які можна отримати за допомогою одного з методів: *fetchall()* (повертає список з усіма рядками), *fetchmany()* (повертає зазначену кількість рядків) і *fetchone()* (повертає один у наборі рядок). Розглянемо отримання даних на прикладі таблиці *people* (рис.2.8).

4.1 Отримання всіх рядків

Наприклад, отримаємо всі раніше додані дані з таблиці *people*:

```
import psycopg2

conn = psycopg2.connect(dbname="example", user="postgres", password="masterkey",
host="127.0.0.1")
cursor = conn.cursor()

# отримуємо всі дані з таблиці people
cursor.execute("SELECT * FROM people")
print(cursor.fetchall())

cursor.close()
conn.close()
```

Під час виконання цієї програми на консоль буде виведено список рядків, де кожен рядок представляє кортеж:

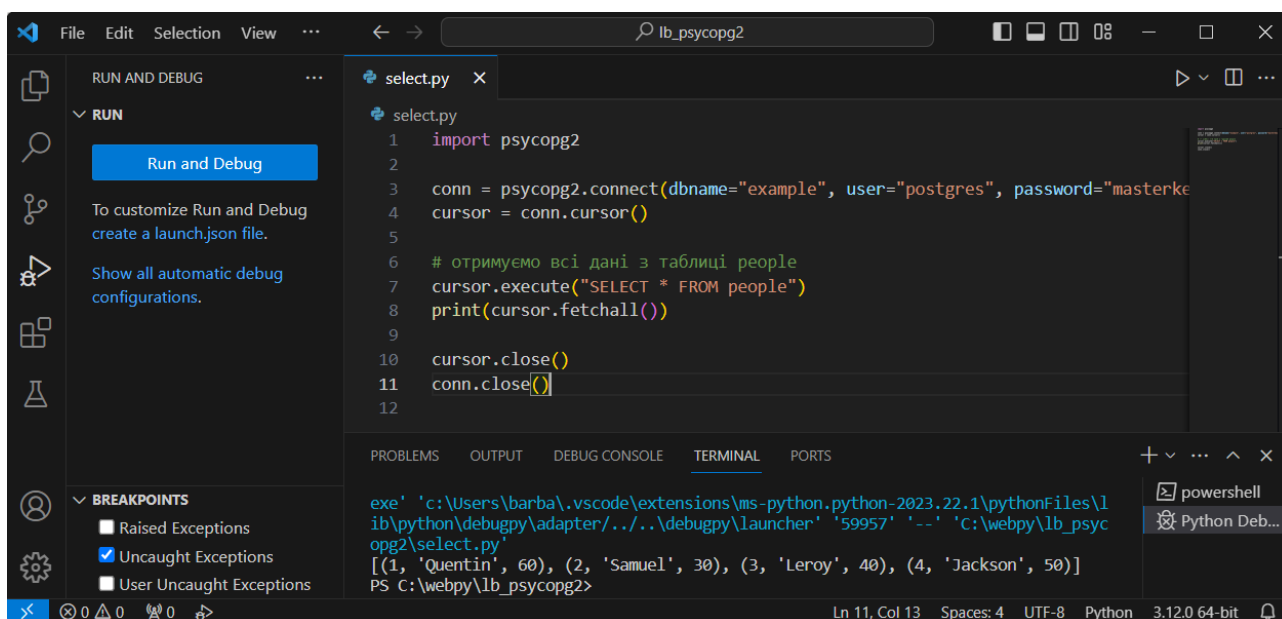


Рисунок 9 – Результат вибірки даних

За необхідності ми можемо перебрати список, використовуючи стандартні циклічні конструкції:

```
import psycopg2
```



```

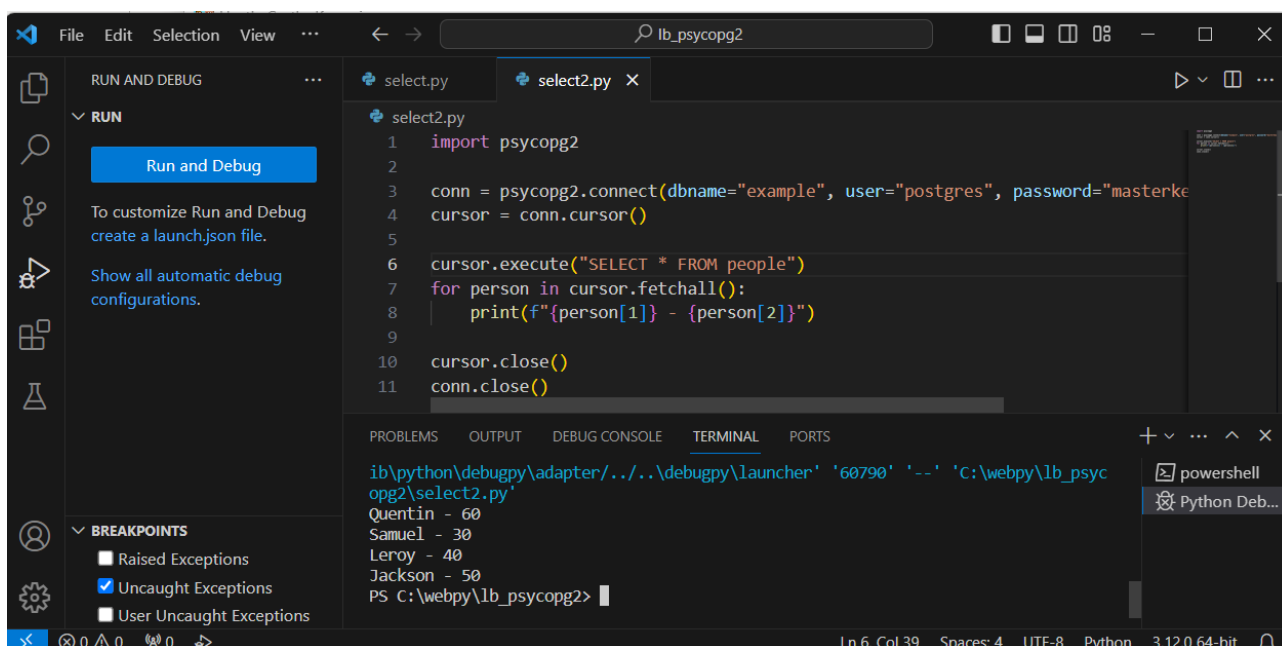
conn = psycopg2.connect(dbname="example", user="postgres", password="masterkey",
host="127.0.0.1")
cursor = conn.cursor()

cursor.execute("SELECT * FROM people")
for person in cursor.fetchall():
    print(f"{person[1]} - {person[2]}")

cursor.close()
conn.close()

```

Результат роботи програми:



Варто зазначити, що в прикладі вище необов'язково викликати метод *fetchall*, ми можемо перебрати курсор у циклі як звичайний набір:

```

for person in cursor:
    print(f"{person[1]} - {person[2]}")

```

4.2 Отримання частини рядків

Отримання частини рядків за допомогою методу *fetchmany()*, у який передається кількість рядків:

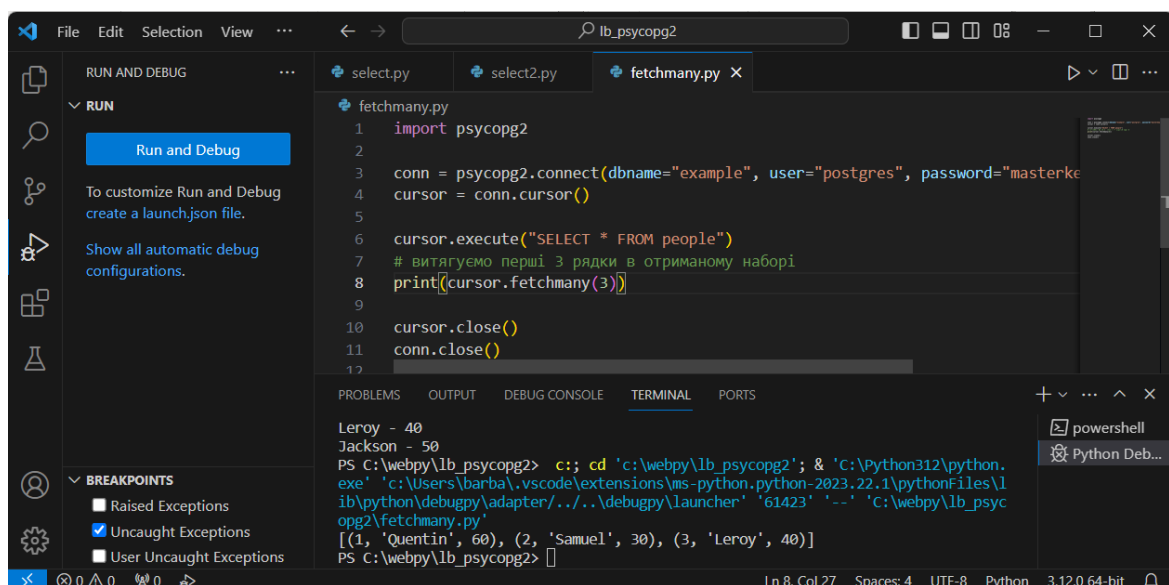
```
import psycopg2

conn = psycopg2.connect(dbname="example", user="postgres", password="masterkey",
host="127.0.0.1")
cursor = conn.cursor()

cursor.execute("SELECT * FROM people")
# витягуємо перші 3 рядки в отриманому наборі
print(cursor.fetchmany(3))

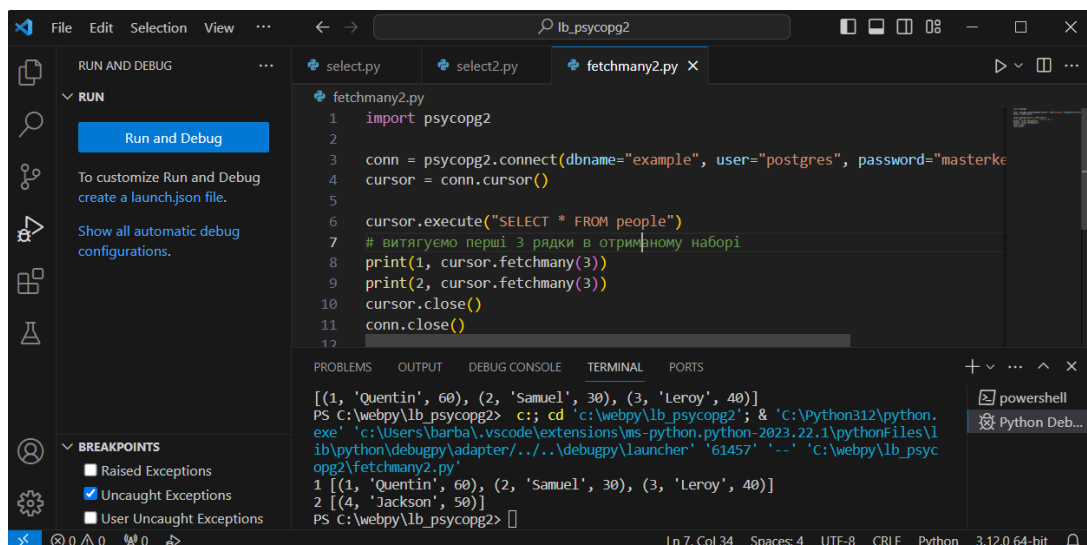
cursor.close()
conn.close()
```

Результат роботи програми:



```
File Edit Selection View ... lb_pyscopg2
select.py select2.py fetchmany.py X
RUN
Run and Debug
To customize Run and Debug create a launch.json file.
Show all automatic debug configurations.
BREAKPOINTS
[ ] Raised Exceptions
[x] Uncaught Exceptions
[ ] User Uncaught Exceptions
fetchmany.py
1 import psycopg2
2
3 conn = psycopg2.connect(dbname="example", user="postgres", password="masterkey",
4 host="127.0.0.1")
5 cursor = conn.cursor()
6 cursor.execute("SELECT * FROM people")
7 # витягуємо перші 3 рядки в отриманому наборі
8 print(cursor.fetchmany(3))
9
10 cursor.close()
11 conn.close()
12
PROBLEMS OUTPUT DEBUG CONSOLE TERMINAL PORTS
Leroy - 40
Jackson - 50
PS C:\webpy\lb_pyscopg2> c:; cd 'c:\webpy\lb_pyscopg2'; & 'c:\Python312\python.exe' 'c:\Users\barba\.vscode\extensions\ms-python.python-2023.22.1\pythonFiles\lib\python\debugpy\adapter\..\..\debugpy\launcher' '61423' '--' 'c:\webpy\lb_pyscopg2\fetchmany.py'
[(1, 'Quentin', 60), (2, 'Samuel', 30), (3, 'Leroy', 40)]
PS C:\webpy\lb_pyscopg2>
```

Виконання цього методу витягує такі раніше невитягнуті рядки:



```
File Edit Selection View ... lb_pyscopg2
select.py select2.py fetchmany2.py X
RUN
Run and Debug
To customize Run and Debug create a launch.json file.
Show all automatic debug configurations.
BREAKPOINTS
[ ] Raised Exceptions
[x] Uncaught Exceptions
[ ] User Uncaught Exceptions
fetchmany2.py
1 import psycopg2
2
3 conn = psycopg2.connect(dbname="example", user="postgres", password="masterkey",
4 host="127.0.0.1")
5 cursor = conn.cursor()
6 cursor.execute("SELECT * FROM people")
7 # витягуємо перші 3 рядки в отриманому наборі
8 print(1, cursor.fetchmany(3))
9 print(2, cursor.fetchmany(3))
10 cursor.close()
11 conn.close()
12
PROBLEMS OUTPUT DEBUG CONSOLE TERMINAL PORTS
[(1, 'Quentin', 60), (2, 'Samuel', 30), (3, 'Leroy', 40)]
PS C:\webpy\lb_pyscopg2> c:; cd 'c:\webpy\lb_pyscopg2'; & 'c:\Python312\python.exe' 'c:\Users\barba\.vscode\extensions\ms-python.python-2023.22.1\pythonFiles\lib\python\debugpy\adapter\..\..\debugpy\launcher' '61457' '--' 'c:\webpy\lb_pyscopg2\fetchmany2.py'
1 [(1, 'Quentin', 60), (2, 'Samuel', 30), (3, 'Leroy', 40)]
2 [(4, 'Jackson', 50)]
PS C:\webpy\lb_pyscopg2>
```

4.3 Отримання одного рядка

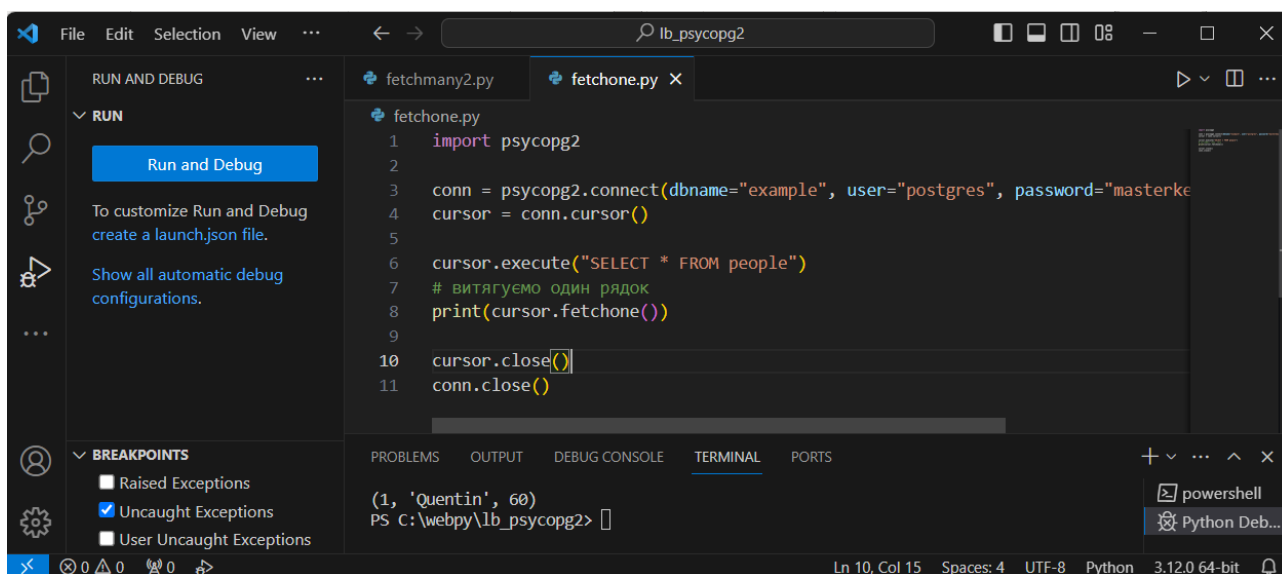
Метод `fetchone()` витягує наступний рядок у вигляді кортежу значень і повертає його. Якщо рядків більше немає, то повертає `None`:

```
import psycopg2

conn = psycopg2.connect(dbname="example", user="postgres", password="masterkey",
                        host="127.0.0.1")
cursor = conn.cursor()

cursor.execute("SELECT * FROM people")
# витягуємо один рядок
print(cursor.fetchone())

cursor.close()
conn.close()
```



Цей метод зручно застосовувати, коли нам треба витягти з бази даних тільки один об'єкт:

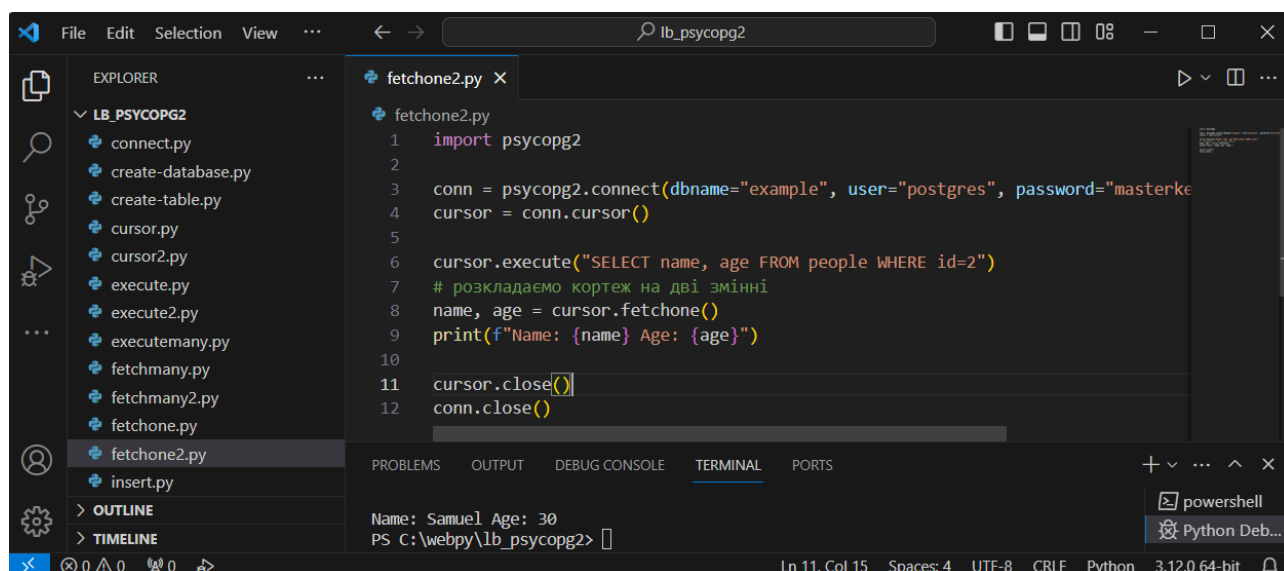
```
import psycopg2

conn = psycopg2.connect(dbname="example", user="postgres", password="masterkey",
host="127.0.0.1")
cursor = conn.cursor()

cursor.execute("SELECT name, age FROM people WHERE id=2")
# розкладаємо кортеж на дві змінні
name, age = cursor.fetchone()
print(f"Name: {name} Age: {age}")

cursor.close()
conn.close()
```

Тут отримуємо з бд рядок з `id=2`, і отриманий результат розкладаємо на дві змінні `name` і `age` (оскільки кортеж у *Python* можна розкласти на окремі значення)



5 Оновлення та видалення даних у PostgreSQL

Розглянемо оновлення та видалення даних на прикладі таблиці *people* (рис.2.8).

5.1 Оновлення

Для оновлення в *SQL* виконується команда *UPDATE*. Наприклад, замінимо у всіх користувачів ім'я з *Quentin* на *Tarantino*:

```

import psycopg2

conn = psycopg2.connect(dbname="example", user="postgres", password="masterkey",
host="127.0.0.1")
cursor = conn.cursor()

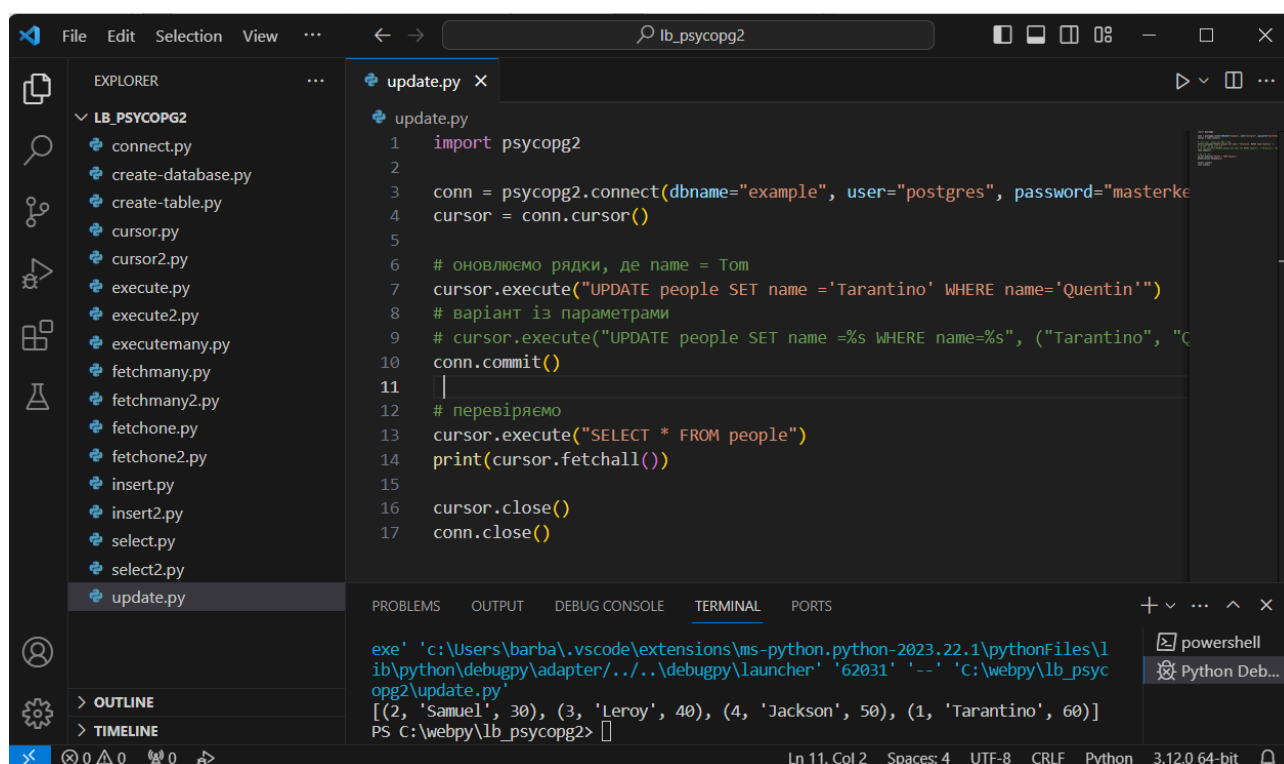
# оновлюємо рядки, де name = Tom
cursor.execute("UPDATE people SET name ='Tarantino' WHERE name='Quentin'")
# варіант із параметрами
# cursor.execute("UPDATE people SET name =%s WHERE name=%s", ("Tarantino", "Quentin"))
conn.commit()

# перевіряємо
cursor.execute("SELECT * FROM people")
print(cursor.fetchall())

cursor.close()
conn.close()

```

Для виконання оновлення також треба виконувати метод `conn.commit()`.



Для множинного оновлення, як і для множинного додавання, застосовується метод `executemany()`:

```
import psycopg2

conn = psycopg2.connect(dbname="example", user="postgres", password="masterkey",
host="127.0.0.1")
cursor = conn.cursor()

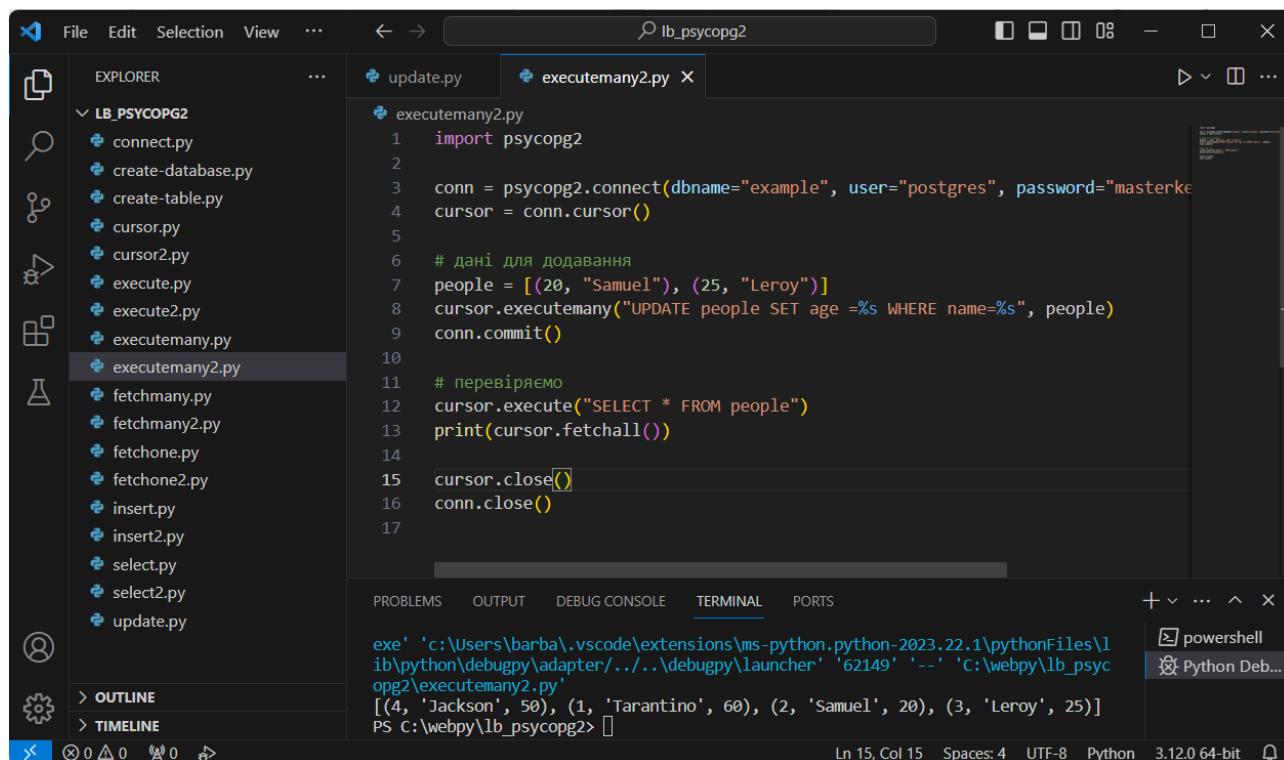
# дані для додавання
people = [(20, "Samuel"), (25, "Leroy")]
cursor.executemany("UPDATE people SET age =%s WHERE name=%s", people)
conn.commit()

# перевіряємо
cursor.execute("SELECT * FROM people")
print(cursor.fetchall())

cursor.close()
conn.close()
```

У цьому випадку на основі списку *people* формуватимуться дві *sql*-команди:

```
UPDATE people SET age =20 WHERE name= 'Samuel'
UPDATE people SET age =25 WHERE name=' Leroy'
```



5.2 Видалення даних

Для видалення в SQL виконується команда DELETE. Наприклад, видалимо всіх користувачів з ім'ям Bob:

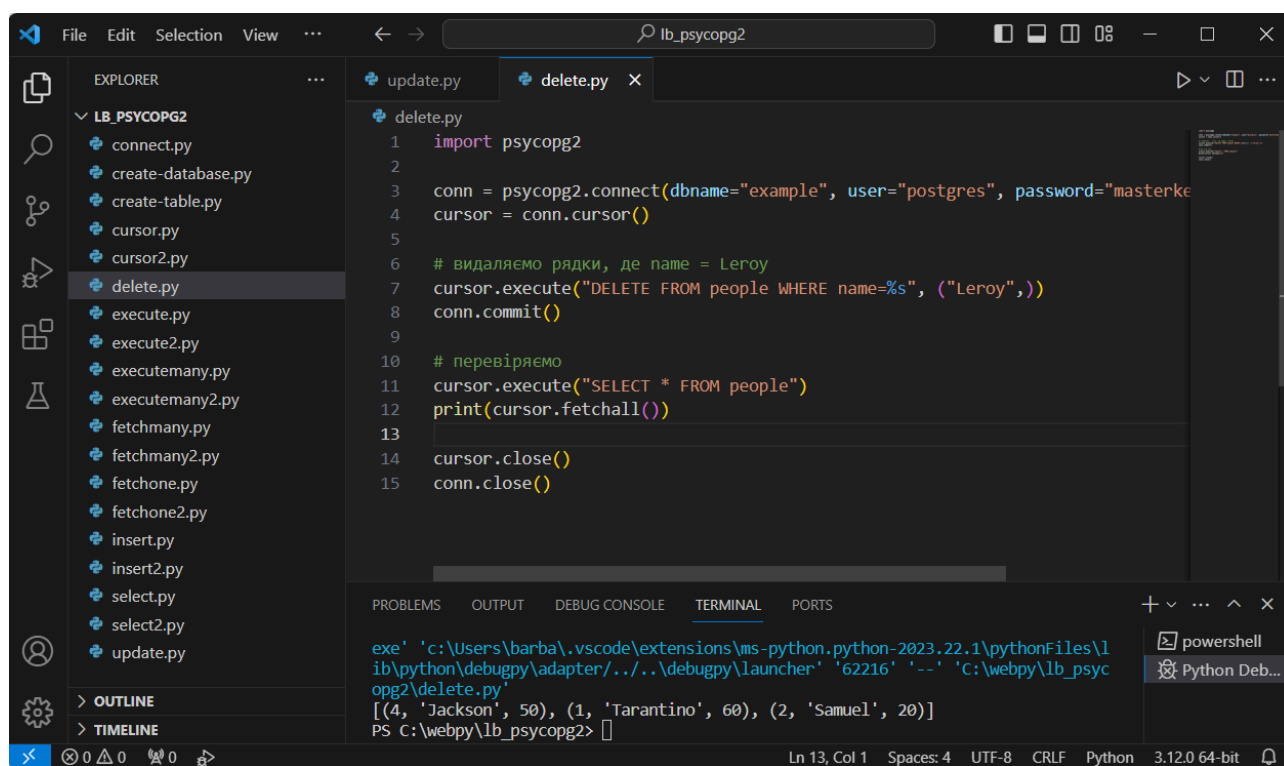
```
import psycopg2

conn = psycopg2.connect(dbname="example", user="postgres", password="masterkey",
host="127.0.0.1")
cursor = conn.cursor()

# видаляємо рядки, де name = Leroy
cursor.execute("DELETE FROM people WHERE name=%s", ("Leroy",))
conn.commit()

# перевіряємо
cursor.execute("SELECT * FROM people")
print(cursor.fetchall())

cursor.close()
conn.close()
```



Множинне видалення:

```
import psycopg2

conn = psycopg2.connect(dbname="example", user="postgres", password="masterkey",
host="127.0.0.1")
cursor = conn.cursor()

# видаляємо рядки з id =2 та id=4
people = [(2,), (4,)]
cursor.executemany("DELETE FROM people WHERE id=%s", people)
conn.commit()

# перевіряємо
cursor.execute("SELECT * FROM people")
print(cursor.fetchall())

cursor.close()
conn.close()
```

